Replicating SWE-Search: MCTS-Enhanced Software Agents on SWE-bench Lite

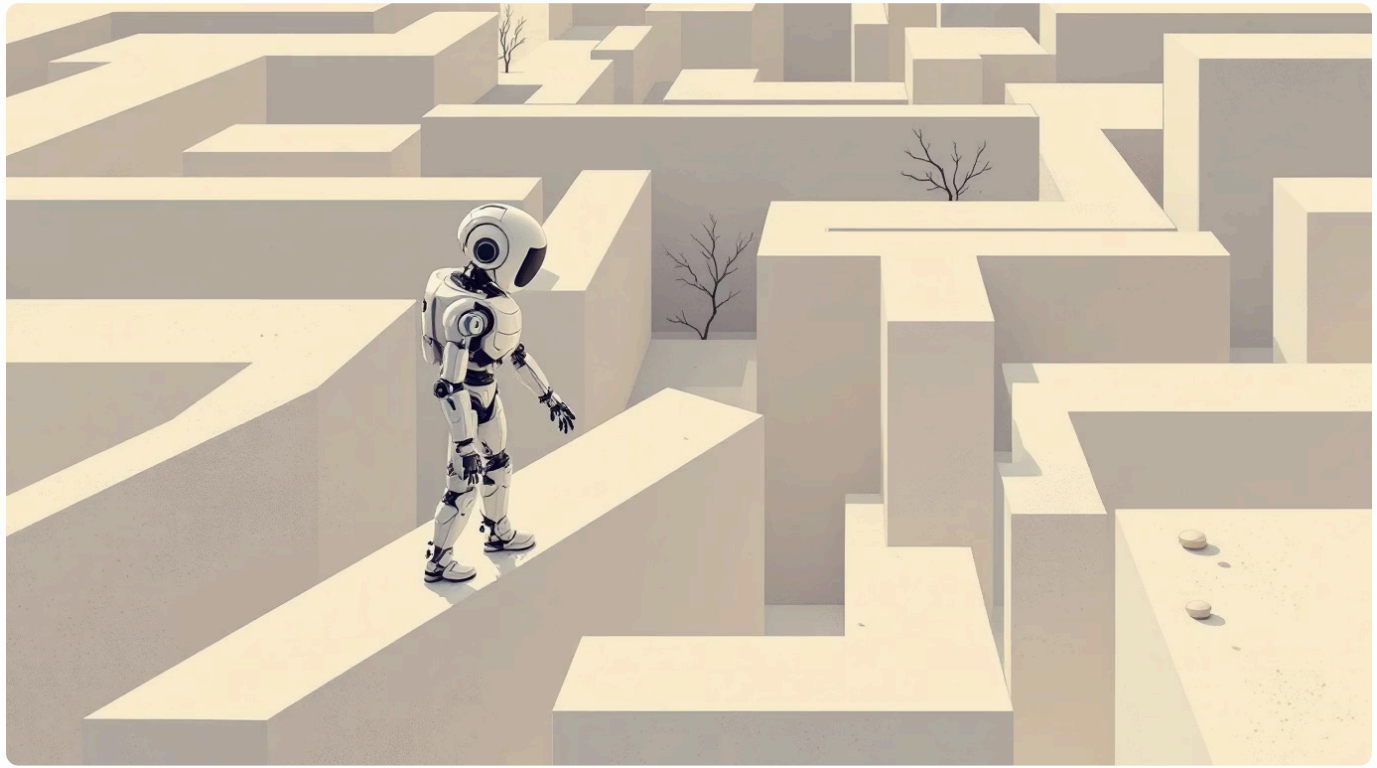
Mayank Bhadrasen, Gaurav Dalal, Srinjana Nag

We replicated a baseline and a proposed MCTS-based agent for automated GitHub issue resolution, validating the SWE-Search framework (Antoniades et al., ICLR 2025)

Made with GAMMA

CORE CHALLENGE

The Linear Agent Problem



Current Limitation

LLM agents follow linear paths without the ability to backtrack or recover from dead ends when fixing real-world bugs. Once a wrong decision is made, the entire trajectory fails.

Impact

This fundamental constraint limits automated issue resolution, forcing agents to get everything right on the first attempt.

Made with GAMMA

SWE-Search: A Multi-Agent Framework

The paper introduces a three-component system that enables strategic exploration and self-correction through modified Monte Carlo Tree Search.

SWE-Agent

Explores a flexible state space with Search, Plan, and Edit actions executable in any order, enabling non-linear problem-solving trajectories

Value Agent

Scores each state AND provides natural language hindsight feedback to guide re-expansion, learning from previous exploration attempts

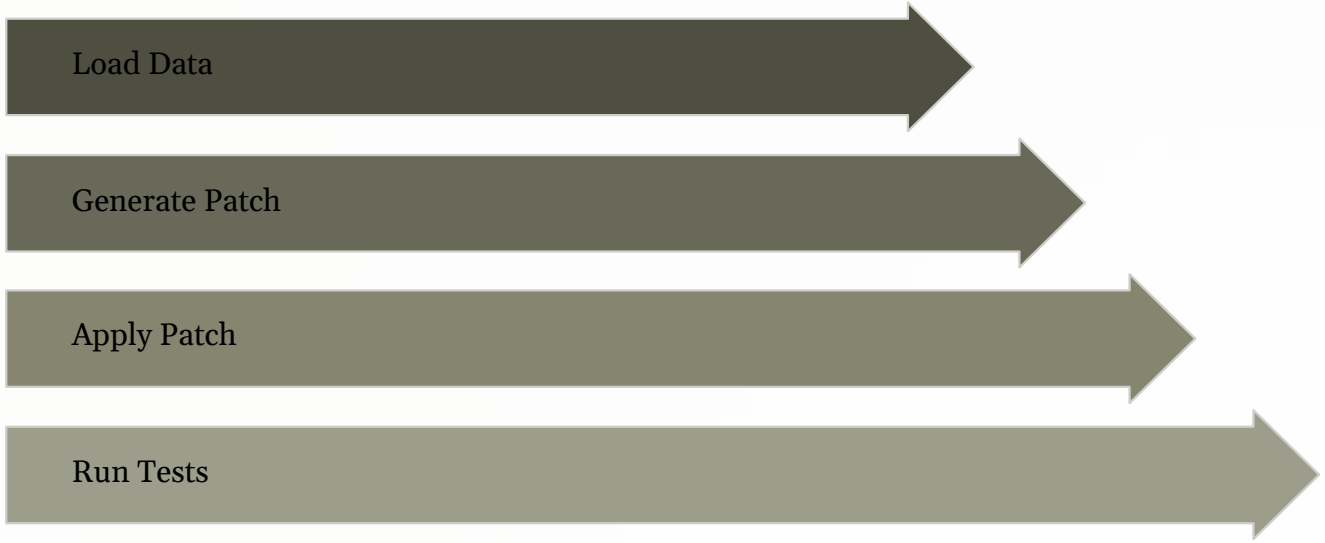
Discriminator Agent

Runs structured multi-agent debate across 3 rounds to select the best solution from up to 5 candidates, improving selection accuracy

Core algorithm: Modified MCTS with custom UCT formula rewarding early exploration while penalizing late over-exploration

Made with GAMMA

Baseline Implementation



Moatless-Adapted Baseline

We replicated the linear agent approach without search capabilities as our reference point.

Pipeline Components

- SWE-bench Lite loaded into SQLite database
- Tool-based agent generates code patches
- Patch application with test execution
- Pass@1 metric: resolved only if tests pass

Minimal dependencies: OpenAI API, no RL frameworks required

Made with GAMMA

SWE-Search Implementation

We built the complete MCTS pipeline on top of the baseline architecture, implementing all three agent components and the search mechanism.

1

MCTS Loop

swe_search/run.py orchestrates node expansion, value scoring, and hindsight feedback injection across the search tree

2

Configuration

configs/swe_search.yaml manages hyperparameters: max iterations (100), max expansions per node (3)

3

Prediction Format

scripts/build_predictions.py formats top solutions into official SWE-bench prediction format

4

Official Evaluation

eval/official_swebench_runner.py executes the official harness for fair, reproducible evaluation

5

Results Aggregation

scripts/summarize_official_eval.py collects and aggregates final metrics across all test instances

Made with GAMMA

Key Architectural Additions

Value Agent

Provides numerical reward plus natural language explanation for each trajectory node

Hindsight Re-expansion

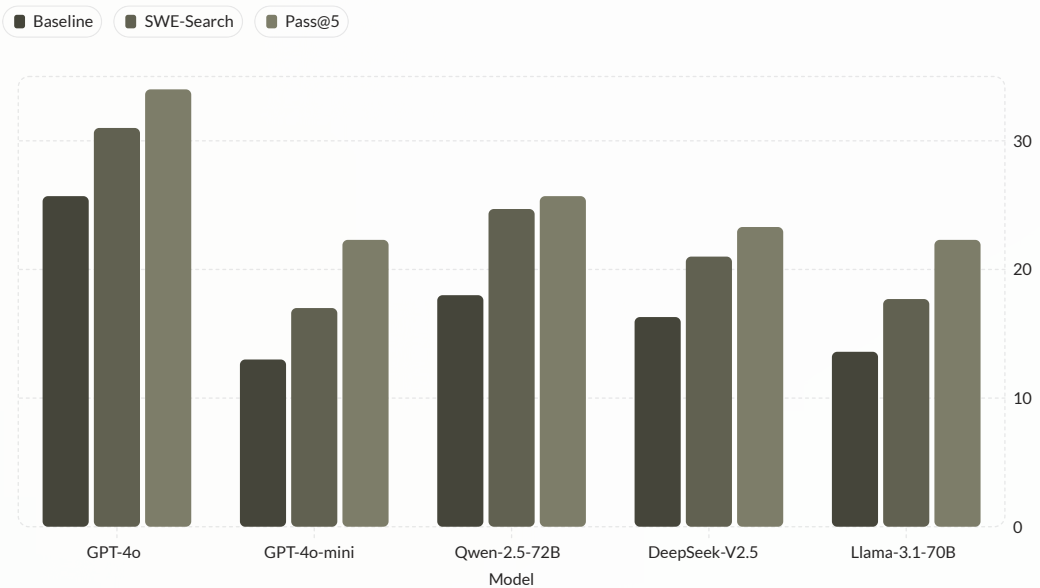
Uses prior feedback to diversify actions during node re-exploration

Discriminator Debate

3-round debate among 5 agents selects best patch from candidate solutions

Made with GAMMA

Experimental Results



Performance Gains

SWE-Search achieves consistent improvements across all tested models, with GPT-4o showing 31.0% resolve rate versus 25.7% baseline.

Average relative improvement: **+23%** across 5 models. The gap between Pass@1 and Pass@5 reveals opportunity for improved discriminator selection.

Made with GAMMA

Validated Key Findings

Inference-Time Scaling

More search iterations consistently resolve more issues, demonstrating effective scaling without model size increases

Value Function Accuracy

Value function alone: 73% accuracy in solution selection; Discriminator raises it to 84% through multi-agent debate

Model Diversity Benefit

Different models solve different subsets of issues — ensemble approaches could further improve overall coverage

Made with GAMMA

Implementation Challenges

14×	$576	$41
Cost Multiplier	SWE-Search	Baseline
GPT-4o runs	300 instances	300 instances

Key Obstacles

- **Computational cost:** Search-based inference significantly more expensive than linear trajectories
- **Environment setup:** Evaluation requires Docker plus SWE-bench harness — non-trivial infrastructure
- **Prompt management:** Faithful hindsight feedback injection demands careful context handling across tree nodes

Made with GAMMA

Conclusion & Future Directions

SWE-Search demonstrates that inference-time search with self-evaluation can significantly improve code agents without larger models or additional training.

1

Validated Architecture

Clean implementation confirms reproducibility with minimal stack: SQLite + OpenAI API

2

Proven Improvement

+23% average relative improvement across 5 models on SWE-bench Lite

3

Scale Search Depth

Explore deeper search trees and longer inference budgets

4

New Applications

Apply framework to vulnerability discovery and security patching

5

Model Ensembles

Combine outputs from multiple models to maximize issue coverage

Made with GAMMA